HYperbolic Promoter Programming Engine (HYPPE) - a foundation model for fungal promoters

iGEM Warsaw 2026

HYPPE: how the model works

A promoter consists of 800 letters from the nucleotide alphabet (A, C, T and G). The answer to the question of how strongly this switch turns a gene on lies somewhere in them.

HYPPE can read that answer. It also writes it back. The model converts a promoter into a short list of discrete symbols. These symbols can be edited and decoded to create a new promoter.

This page explains the model twice. Each section begins with a plain-language explanation of the idea. Then it presents the main mathematics behind this idea. The final sections cover how we selected the ten synthetic promoters for the wet lab and the limitations of our data.

What came out of this, in four findings

The sections below build the model and then take it apart. This one says what the work produced, because four of the findings are about the organism rather than about the architecture, and two of them changed what we built. Each is derived in full further down, with the control that had to fire before we were willing to report it.

Every number here rests on measurements of real promoters, 2,332 of them

Nobody has measured promoter strength for thousands of Trichoderma genes, so the label is borrowed rather than direct: RNA-seq reports transcript abundance, and a promoter working hard generally yields more of it. What we could control is whether those measurements are comparable with each other. Every run was requantified from its raw reads with identical settings rather than taken from a deposited count table, because a deposited table carries the choices of whoever produced it and stacking tables stacks the choices. The experiment behind each run was then read by hand, run by run, in three passes, because public archives record what was done to a culture in free text with no convention and often record nothing at all. The corpus that came out is 2,332 publicly deposited runs across twenty reference genomes, and no measurement of our own and none from the target strain P1 is anywhere in it. Section 1 is how the labels were built, section 6 is how they were split so that near-identical orthologues could not sit on both sides of a test.

What the model told us about promoters as a class of part, and about pks1 in particular

Within one species the promoter cannot explain a difference in expression. Promoters of a single gene are 98.7% identical across the ten T. atroviride strains while the proteins they drive are 99.4% identical, against a shuffled-sequence background near 35% that any pair of unrelated promoters reaches by chance. The sharpest case is the gene this project is about: six strains share the pks1 promoter byte for byte while their measured expression spans 0.039 to 7.624, a factor of 195. Whatever separates them, it is not the 800 bases in front of the gene.

That measurement is why the model conditions on strain at all, and it is also what exposed the worst thing we shipped. The first strain tree had ten leaves against a registry of twenty, so thirteen strains collapsed onto one coordinate, six of them inside that pks1-identical set. The channel was not dead; it was structurally unable to express the one contrast we most needed, and no loss curve would ever have shown it. The replacement agrees with an independent reference tree at Pearson +0.9953 over 21 shared pairs and takes collapsed pairs from 78 of 190 down to 3. Section 3 shows where the twenty genomes now sit, including the uncomfortable half of that result: the embedding learned the species boundary, which costs 2.8 times more to cross than the widest gap inside the species, and very little below it. The cycle itself is D4 on the Engineering page.

The result we would put forward as the strongest: a binding site the model found on its own

The model recovered a real regulatory element from nothing but its own behaviour. Ranking every six-letter word by how much of the model's sensitivity sits beneath it, and comparing each word only against the other arrangements of its own letters so that base composition cannot manufacture the answer, the strongest preference it holds is CACGTG. It comes first of the 159 arrangements of its own composition, reads the same on both strands, and its two flanking variants rank tenth and fourteenth out of 3,770. That is the E-box, the binding site of basic helix-loop-helix transcription factors, and no motif database entered the analysis at any point.

The controls matter more than the result. The words that top the raw ranking are simply GC-rich and none survives comparison with its own arrangements. Chromatin-accessibility footprinting in Aspergillus niger independently recovers the same element among the five most enriched motifs in open promoter regions, beneath genes enriched for ribosomal proteins and translation, which is the most highly expressed class a fungus has and exactly what a model of promoter strength has reason to learn. Sliding a six-base window along that ten-base footprint, the three windows containing the core rank fifth, tenth and fourteenth, the fourth falls to 155th, and the fifth, which has lost the core, falls to 2,915th. The same analysis also turned up a sense of position nobody specified: a substitution matters roughly three times more in the last 160 bases before the start codon than at the far end of the window, although the architecture supplies only a single

parameter-free ramp. Both results are under What the model learned about the sequence itself, in section 7.

What another team can take from this: the limit we hit, and the experiment that lifts it

What it did not learn is as informative, and we can say why it could not have. The canonical regulators of fungal promoters sit in the middle or at the bottom of that same ranking: carbon catabolite repression, the CCAAT complex, the xylanase activator and the light response elements. The last of those is the one thing pks1 was predicted to show, since the metabolite it makes is produced in darkness and suppressed by light. The reason is the corpus rather than the architecture, and it is not the reason we first wrote down. Sequence variation is not short: 61% of orthogroups appear in two or more strains, and within the single control condition nineteen strains share 15,930 genes, 88% of which differ in sequence.

What is short is the conditions that would teach a response. Light is measured in three of nineteen strains and in 144 of 2,332 samples, and mechanical injury in one strain. Worse, the only contrasts available are confounded, because two strains differ in their regulators as well as in their promoters, and the strain channel exists to absorb exactly that. Learning which base makes a promoter responsive needs variation at a fixed genetic background, which means a mutagenesis library in one strain measured under both conditions. That is a concrete experiment and it is the one we would run next. These negatives are measurements rather than silence: on the same promoter and against the same null, the most sensitive five per cent of positions beat chance by about nineteen standard deviations. Section 9 states what the whole table does and does not permit.

How the modelling changed the project, including the day it failed at the only locus that mattered

An average cannot answer a question asked at one locus. Across the corpus the model ranked at 0.569, which looked like enough. On held-out orthologues of pks1 it ordered the family at -0.49, worse than random rather than merely weak, and the decision the project actually needed, whether a sequence sits above pks1, came out at 0.5669 balanced accuracy on the 299 P1 validation promoters, which is a coin toss. The corpus number hid it because that family is a few dozen rows inside a corpus of thousands. The failure sat precisely where the project needed the model to work, since every design brief is written relative to pks1.

The repair was deliberately minimal and its cost is reported rather than buried: the ranking head alone, fine-tuned for 225 steps, taking family ranking from -0.49 to +0.89 and the above-pks1 decision from 0.5669 to 0.8138, while corpus-wide ranking slipped from 0.569 to 0.514. It is kept as a separate checkpoint so that the model proposing a sequence and the model judging it are not the same object, and that was verified rather than assumed: of 471 tensors, 465 are identical bit for bit and the six that differ belong to the ranking head. Their agreement on the delivered panel is therefore a result and not an artefact of

whichever head happened to score. Cycle D6 on the Engineering page is that cycle, section 8 is the ten sequences it made possible, and section 9 is what those predictions claim and what they do not.

1. The shape of the problem

There are three things that make promoter strength difficult to learn. Each one influenced a design decision.

The label is noisy and borrowed. Other laboratories measured the expression. These studies were never intended to be compared with each other. We pool them and correct the between-study component. The remaining results still carry the spread of the original experiments. Therefore, the model predicts a mean and its own uncertainty. We judge it based on ordering first and level second.

The examples are not independent. One gene has orthologues in twenty related strains. They share most of their promoter. If we split them at random, the model is tested on near-copies of its own training data. The score looks good. However, it does not survive on a new locus. So our split groups whole families together.

A dense vector cannot be edited. Suppose a model makes accurate predictions but stores a promoter as 512 anonymous floating-point numbers. This tells a biologist nothing about where the strength lies. It offers no guidance for designing a new sequence. Therefore, our representation is discrete and hierarchical. Coarse symbols represent broad positions. Finer symbols hold detail. Each one is tied to a length in base pairs.

What we assume, and why

This design is based on five assumptions, each of which is tested later on this page. The first assumption is that pooled expression data from many different studies becomes comparable once the between-study component is removed. This is important because the labels are borrowed and nobody has measured this corpus for us. The second assumption is that a promoter's codes must not depend on the growth condition because, otherwise, editing those codes would not provide information about the sequence, and the design loop would have nothing to guide it. This assumption is enforced by the architecture rather than by discipline, and it is the first crossed path in Figure 1. The third assumption is that the hierarchy against which we quantify is a tree because we imposed one, not because the sequence itself is tree-shaped. We measured the tree-likeness of regulatory DNA and found that it was no more tree-like than random points. Therefore, Section 3 states this against our own interests. The fourth assumption is that orthologues of one gene are not independent examples since they share most of their promoter. A random split would test the model on near-copies of its own training data. This is what the split in Section 6 is based on. The fifth point is that the phylogeny acts as a prior on the strain space rather than as a constraint. While 439 protein markers are used to place the strains, expression may disagree and should be given precedence. Cycle D4 on the Engineering

page presents a case where expression did not prevail, necessitating the manual adjustment of the prior.

The wet lab is the test, not the input. None of the measurements are ours; the corpus comprises public RNA-seq data from other laboratories. The P1 target strain is included in the data, and we mention this because it is easy to assume the opposite: 24 of its samples are included, and these samples carry a measurement in 10,168 training orthogroups. The model has never seen a designed sequence, and both anchor promoters are excluded from training. The strain enters through its learned position on the phylogeny; the condition, through its written description; and the plasmid context, through a learned UNKNOWN token. We state this clearly because it has two implications: the ten designs are a zero-shot prediction with respect to the design task, and the model has never encountered the assay it is about to be evaluated against. Section 9 lists the three domain shifts between our label and a wet read-out.

2. The architecture

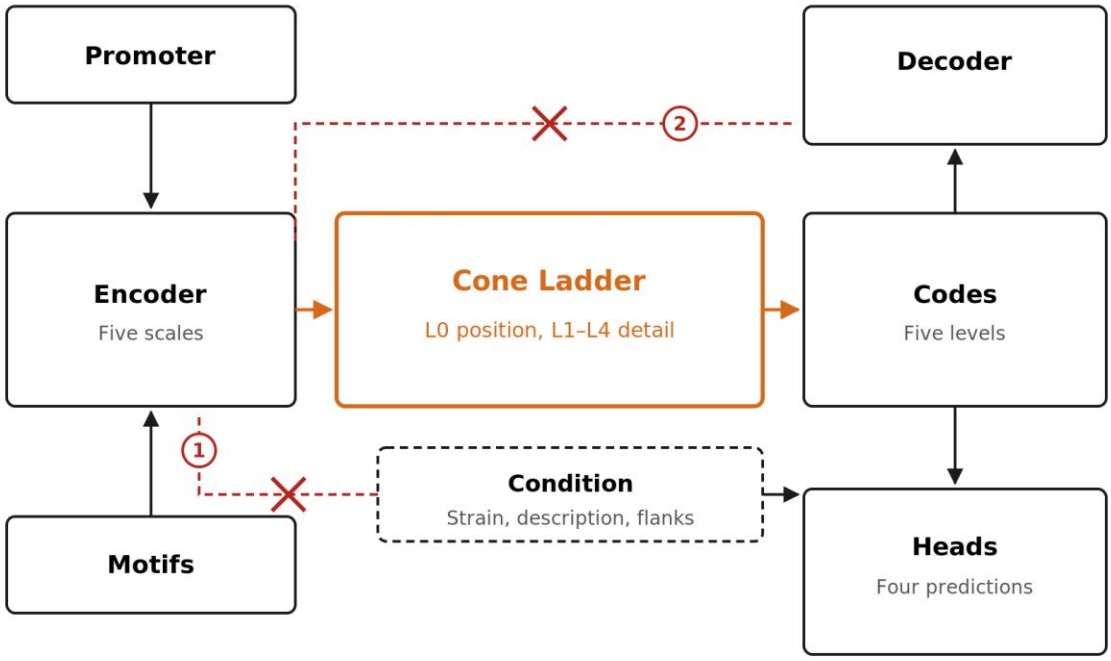


Figure 1. The architecture of the HYPPE model. A promoter and its motif occupancy go into the Encoder. The Cone Ladder then turns this into discrete codes, which feed both a decoder that rebuilds the sequence and the heads that predict how strongly they fire. The two crossed paths represent the choices that render the codes usable for design purposes, and the code enforces both of these. The growth condition never reaches the encoder, meaning that a promoter always receives the same codes, regardless of the experiment. Furthermore, nothing but the codes reach the decoder, meaning that anything they fail to carry is visibly lost rather than quietly smuggled through.

This shows where each piece comes from. The hyperboloid and its optimisation follow the work of *Nickel and Kiela (2018)* and *Bécigneul and Ganea (2019)*. The level-0 codebook is a VQ-VAE codebook, as defined by *van den Oord et al. (2017)*, while the finer levels employ the finite scalar quantisation method of *Mentzer et al. (2024)*. The decoder is constrained by the optimisation scheme of *Rezende and Viola (2018)* rather than by a manually set loss weight. The strength head uses the beta-NLL objective of *Seitzer et al. (2022)*, the ranking head uses the *Bradley and Terry (1952)* likelihood and the condition's description channel uses the sentence embedding of *Wang et al. (2020)*. Full references are provided at the end. The genomic flanks are read by the DNA language model of *Ji et al. (2021)*, and the description channel of the delivered model uses the biomedical sentence encoder of *Liu et al. (2021)* rather than the general-purpose embedding of *Wang et al. (2020)* that earlier versions used; the subsection below says how both were run and what was measured about them.

These two elements represent the architecture's load-bearing choices:

1) **The decoder only sees codes.** Suppose the decoder could also read the encoder’s continuous features. It would use these features and ignore the codes, rendering the codes decorative. We measured exactly that on a sibling model. With a continuous bypass open, the reconstruction score was 0.9984. Reconstruction using the codes alone produced a reading of 0.29, compared to a chance floor reading of 0.26. It is closing the bypass that makes 'decoding the codes into a sequence' a real operation.

2) **The condition never reaches the encoder.** The strain, growth conditions and neighbouring DNA all affect how strongly a promoter fires. However, they are not part of the promoter itself. We keep them behind the quantiser. The code of a sequence is therefore a function of the sequence alone. The same promoter has the same codes in every strain and only the condition moves the prediction.

Where the condition labels come from, and how the model reads them

Each expression value belongs to a cell defined by three coordinates: an orthogroup, a genome and a condition. The first two are produced by the upstream pipeline. The third does not. A public archive records what was done to a culture in free text. The person who deposited the run wrote it in no fixed format and at whatever level of detail they chose.

Often, nothing was written. As authors share no convention, one treatment may appear as a sentence or a three-word fragment, or not at all. As no program can read that, we must read the metadata of every run by hand.

The work was carried out in three stages. The order was important because each pass changed the scheme used in the next one.

The first pass covered Trichoderma atroviride alone. For each run, we collected all the information available on its NCBI and ENA pages. From this material, we established an initial set of distinct categories: control, biotic antagonism, light response, mechanical injury, plant association and others. These categories were derived from the content of the records, rather than from our own expectations. We also recorded the genetic background. Each run used either a wild type or a strain carrying a knockout or an overexpression, and we noted the relevant genes.

The second pass applied the same logic to every Trichoderma species in the corpus. For each run, we recorded the external conditions in plain text, assigned them to a group and annotated the genetic background. Widening the scope revealed a new category: an unusual carbon source. We then revised the earlier assignments against this new category. The third pass normalised the text so that every description followed the same convention. This step is more important than it sounds because the text is what the model reads.

The result is a hand-curated table containing details of all 2,332 publicly available Trichoderma RNA-seq runs. Each run has a plain-text condition belonging to one of seven categories and a genetic annotation. These categories are control, carbon source, biotic antagonism, light response, plant association, mechanical injury and other. This order cannot be changed because it indexes the output columns of the strength head; adding a category would therefore alter the entire list. Reordering would change the meaning of every column simultaneously.

The text is the channel through which the message is conveyed. The category is the column. The label is an average of the samples behind one cell. In this model, a promoter therefore has no single strength: it has one per condition. The strength head produces a column for each of the seven categories, plus the genome marginal. The direction of a column is not a free parameter, but is generated by pushing the English description of that category through one shared projection, meaning the descriptions determine the direction of the columns.

We considered the obvious alternative before rejecting it. The first thing anyone would try is feeding the categories in directly as a seven-way multi-hot. Under the study split, it reaches 0.106; a 64-dimensional vector of text reaches 0.422; and text outperforms the categories by between 3.6 and 4.5 times under both splits we tested. For P1, the strain that the wet lab will test, the categories perform worse than uselessly at −0.0115, whereas text achieves +0.1812. The categories can account for at most 24.81% of the reproducible conditional signal. The remaining 75.19% separates only below the category level. This is

inside groups our own scheme calls identical. Therefore, the seven categories never enter as seven input dimensions.

Two consequences follow. First, regarding curation rather than modelling, a mediocre free-text description is worth more than a perfectly assigned category. Arguing about where a boundary falls is mostly wasted effort. Ensuring that every run contains some faithful sentences is not. Secondly, an eighth category costs nothing. It is named in English, processed by the same frozen encoder and added as a row in a buffer; the parameter count remains unchanged. This model is expanded by describing a new experiment type.

So, the genetic annotation isn't actually a feature. It never reaches the model, although it would be easy to assume that it does. It is used earlier on as a filter. The sample policy discards 793 runs with a mutant background and 41 controls before anyone sees a row. This leaves 1,498 runs, 1,487 of which carry a usable description. The reason is biological. Expression measured in a strain whose regulator has been deleted does not reflect that promoter in a wild type. Pooling the two would attribute the effect of the knockout to the promoter's label, meaning the model would have to explain it using a sequence that never changed. It is only by recording the background by hand that dropping those runs is possible, so the annotation earns its place by what it removes.

The models we borrowed, and how their vectors were created

Two of the inputs in Figure 1 are not learnt here. The genomic neighbourhood of a locus and the written description of an experiment are provided as vectors from published language models. Both are frozen. Each is run once over the corpus, the output is written to a cache and the training loop reads the cache. No gradient reaches their weights. Their values do not change between epochs. Neither of them ever sees the promoter. Only our own encoder reads the 800 bases.

The reading of the flanks is conducted by **DNABERT** (*Ji et al., 2021*). It is a transformer model from the BERT family (*Devlin et al., 2019*) that has been pre-trained on the human genome and 134 other genomes, including one from fungi. It interprets DNA as sequences of six consecutive letters. Each locus comprises two sequences of neighbouring DNA, each 1,000 bp long, on either side. A 1,000 bp flank produces 995 overlapping six-mers, exceeding the hard limit of 512 tokens. Truncating them would discard half of each flank. Instead, we cover each flank with two overlapping windows. The last window is slid to the end of the flank, rather than being left wherever the stride happens to land. This ensures that the junction with the locus is always visible. This junction is the informative part. Each window is averaged over the final layer under the attention mask to prevent padding from diluting the vector. The window vectors are averaged to create a summary containing 768 numbers. The cache holds 477,253 flank rows and 238,831 promoter rows in float16 format, covering 99.99% of the corpus. Rather than resolving discrepancies through guesswork, 921 identifiers that were listed twice with conflicting vectors were dropped.

This is version 1 of the model due to environmental factors, not version 2. On the training machine, both DNABERT-2 (*Zhou et al., 2024*) and the nucleotide transformer fail to load

under Transformers 5.10.2. Their remote code is incompatible with the installed library, and attempting to patch it results in a meta-device error. DNABERT v1 with k = 6 loads as a plain BertModel with no remote code. Measurements show that we lost little. Using a linear probe on 5% of the corpus, the plain 6-mer composition of the same sequence scored 0.5313 against 0.5125 for the mean-pooled DNABERT-2 vector. This is a tie at a threshold of 0.0333, which was set prior to the comparison. A larger DNA language model would have been preferable.

We tested the capacity of these vectors before trusting them. Mean-pooled transformer embeddings are known to collapse into a few directions, and these do too. The effective rank of the averaged vectors is 13.9 out of 768 for flanks and 14.7 for promoters. Two random flanks have a mean cosine of 0.958. A flank and its reverse complement agree at 0.974, meaning the encoder is almost blind to orientation. Therefore, we give the channel no room that it cannot use. It passes through the same 16-number bottleneck as every other conditioning channel in Figure 6. In a quarter of the training rows, it is replaced by the learned unknown token. It is this second choice that makes design possible. The pHYPPE plasmid neighbourhood sits 5.1 standard deviations outside the genomic flanks observed during training. At design time, it reaches the model. It does so as the UNKNOWN token. This is a state the model was trained to work in.

A special computer program called **SapBERT** (*Liu et al., 2021*) reads the experiment description. SapBERT is a further trained version of PubMedBERT (*Gu et al., 2021*) that pulls together synonyms of one biomedical entity. This approach was selected so that 'mycoparasitism', 'conidiation', and 'carbon catabolite repression' would be recognised as entities rather than character sequences. The text is pooled using a masked mean. We then subtract this mean vector from all 2,332 cached description vectors and re-normalise them to a unit length. This centring step is a measurement, not an embellishment. Raw SapBERT is anisotropic; the seven condition descriptions have a mean cosine similarity of 0.7153 and a maximum similarity of 0.8713, indicating that they are almost interchangeable. After centring, the maximum off-diagonal cosine is 0.2135. The pair closest to the mean is 'biotic antagonism with plant association', which is consistent with a biologist's prediction. Prior to taking the measurement, we set the selection rule to the lowest maximum off-diagonal cosine and rejected any encoder above 0.95.

The vectors are cached per conditioning cell and are keyed by the combination of the **genome** and **category**. This is also the key that the corpus rows are joined on. The artefact holds 85 keys: 66 category cells and 19 genome marginals. It is a unit-norm matrix measuring 85 by 768, and all 740,900 measured targets correspond to a key. Keying by cell rather than by row is a measurement in itself. For the per-sample vectors, before any aggregation, the η^2 value for the 19-level genome factor is 0.1348 and the η^2 value for the 66-level cell factor is 0.3317. The cell key retains two and a half times more text variance for the same single token. There is no fallback from a missing cell key to the genome marginal. A missing description never becomes 'text_proj(zeros)', which would be a

learned constant reporting a share of the context while carrying no information. Instead, it reaches the model as a learned unknown token, flagged and counted.

Two further models were considered and are deliberately absent. Fine-tuning either borrowed encoder on our corpus has not been measured here, and we do not claim to have done so. The k-mer composition of the promoter itself does not enter the model at any point. It is a deterministic function of the input, so it adds no information and only modifies the inductive bias. The surplus it shows on a linear probe has been measured three times in this project and found to be not transferable end to end.

3. Why hyperbolic?

Hyperbolic space is the natural home of trees. The number of points at a given distance from a centre grows exponentially with that distance, and the same is true of the number of nodes at a given depth in a branching tree. In contrast, Euclidean space grows polynomially, meaning that a deep tree embedded in it crowds the leaves.

The usual argument for using hyperbolic space to model biological sequences is that the data are hierarchical. However, we found that this argument does not survive measurement. We used the standard measure of tree-likeness. We applied this to a regulatory sequence corpus and a motif bank in the sibling track of this codebase, rather than the Trichoderma corpus. Regulatory sequences are not meaningfully more tree-like than random points. The motif space was found to be less tree-like than random Euclidean points.

The code hierarchy we impose is the one thing here that is a tree by construction. There is one coarse symbol, and under it are the refinements that specialise it. It is this, and this alone, that makes the geometry hyperbolic. We phrase it this way because the opposite claim would be easy to make and false.

The hyperboloid

We work on the upper sheet of a two-sheeted hyperboloid. Take the Minkowski form on $\mathbb{R}^{n+1}$:

$$\langle x, y \rangle_{\mathcal{L}} = -x_0 y_0 + \sum_{i=1}^{n} x_i y_i,$$

The model space is

$$\mathbb{H}^n = \{ x \in \mathbb{R}^{n+1} : \langle x, x \rangle_{\mathcal{L}} = -1, x_0 > 0 \},$$

and the distance between two of its points is

$$d(x, y) = \operatorname{arcosh}(-\langle x, y \rangle_{\mathcal{L}}).$$

Three operations do all the work. The first moves from a point p along a geodesic, with initial velocity v in the tangent space T_p:

$$\exp_p(v) = \cosh(\|v\|_{\mathcal{L}})p + \sinh(\|v\|_{\mathcal{L}})\frac{v}{\|v\|_{\mathcal{L}}},$$

The second is its inverse $\log_p(q)$. It returns the velocity that takes you from p to q. The third is parallel transport $\mathrm{PT}_{p\to q}(v)$. It carries a tangent vector from one point to another without rotating it.

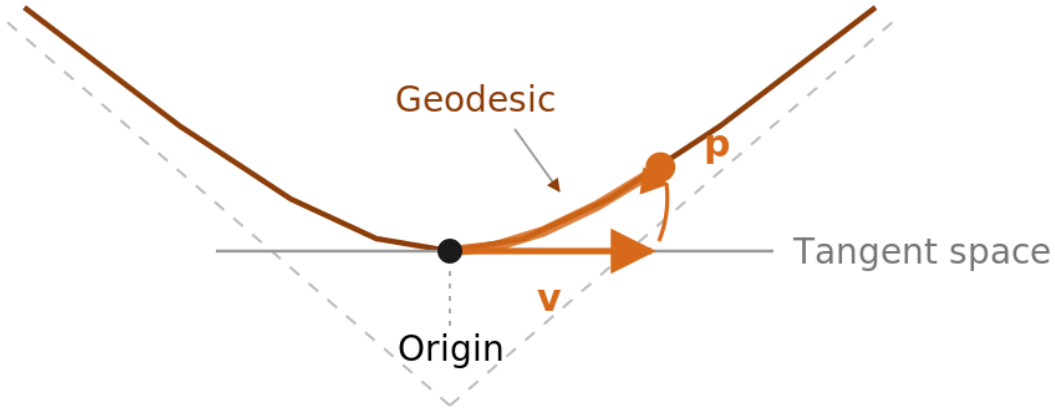


Figure 2: How a flat vector becomes a point on a curved surface. The network produces a standard straight vector v, which the exponential map then wraps around the hyperboloid as a curve of the same length that ends at p. This is the sole connection between the flat space that the network operates in and the curved space that the Codes inhabit, and each step of the Cone Ladder represents one application of it.

Transport is not an improvement on the method. A vector that is tangent at the origin cannot be tangent elsewhere. It is transport that makes the composition in the next section type-correct.

The one place the geometry holds data rather than codes

The ladder is hyperbolic because we impose a tree on it. The strain registry is the other hyperbolic object in the model, and it is the one whose tree was not invented by us: twenty genomes sit on $\mathbb{H}^3$, pulled toward the patristic distances of a 439 protein markers phylogeny by a prior the data is allowed to overrule.

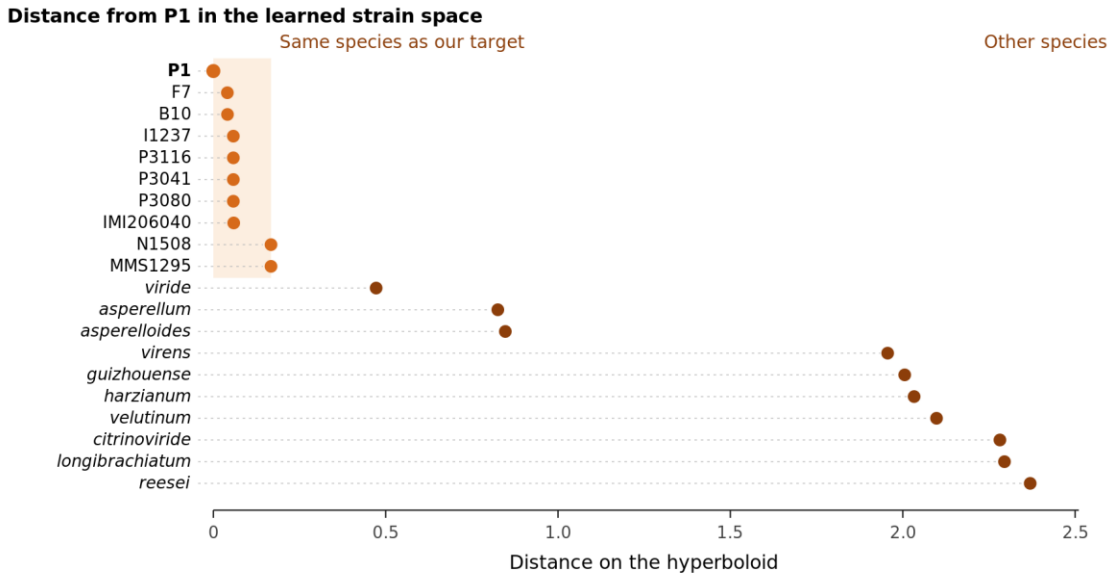


Figure 3: Shows where the model places each of the twenty genomes, as read straight from the delivered checkpoint. The ten T. atroviride strains, including our design target, P1 - are all within 0.167 of each other. The nearest other species is at 0.472, and the furthest is at 2.370. Therefore, the model has clearly learned the species boundary and very little below it. This is the shape that a relational embedding is meant to have, and this is also why a plain lookup table outperforms it when it comes to distinguishing between strains it has already seen.

There are two readings, and the second is the uncomfortable one. The ten T. atroviride strains, including our design target P1, are within 0.167 of each other. The nearest other species is 0.472 away, and the furthest is 2.370 away. Therefore, crossing the species boundary costs at least 2.8 times more than the widest gap within the species. The embedding has learned the taxonomy.

However, it has not learned much below this level. Six of these strains share the pks1 promoter byte for byte, yet their measured expression ranges from 0.039 to 7.624. A geometry that places them 0.06 apart cannot be the sole distinguishing factor. We measured the value of the alternative by holding each strain out in turn and refitting. A flat one-hot model gains +0.032 on a strain it was fitted to and loses −0.007 on a strain it has never encountered. The entire value of the embedding is memorised separation, and P1 is inside the fitted set rather than outside it. Therefore, on the strain we design for, this incurs a cost rather than a gain. We retain the embedding for strains that the pipeline has not encountered and report the loss incurred on strains that it has.

4. The cone ladder

Here is the central idea in one sentence: **Level 0 says where the promoter is, and every later level says how to adjust that.** The adjustments come from a finite number of codes.

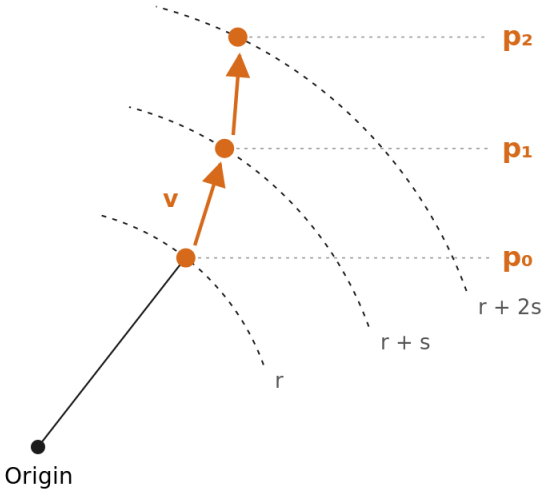


Figure 4. This shows how the ladder is built. The coarse code places p_0 on the inner shell; this is one of 128 learned points. Each finer level adds a short, bounded step from its parent, starting with p_1 and continuing down to L4. Each step also moves the point one fixed increment further out, so depth in the hierarchy literally corresponds to radius in the space. Without this, the radius fluctuates and all five levels settle onto a single shell, rendering the terms 'coarse' and 'fine' meaningless.

Write c_k for the code chosen at level k:

$$p_0 = \text{codebook}[c_0],\ p_{k+1} = \exp_{p_k}(\text{PT}_{o \to p_k}(v_{k+1}) + s_k \text{radial}(p_k)),$$

Here o is the origin. The vector v_{k+1} is the quantised tangent vector of level $k+1$. The step $s_k = 0.15$ is the fixed radial increment. And $\text{radial}(p)$ is the unit tangent at p pointing straight away from the origin.

Two properties follow, both are used downstream:

1) **Containment.** An aperture caps the tangent step, so a child can only be so far from its parent. Coarse codes therefore constrain fine ones. That is what “hierarchy” has to mean in practice.

2) **The prefix property.** Every refinement grid contains *exactly zero*. A level can say “no change”. So the ladder can be truncated after any level and still give a legal code that decodes to a sequence. That is why we can ask what a promoter looks like with only its coarse structure kept, and get an answer rather than an error.

Quantisation

Level 0 picks from a learned codebook of 128 points on **the manifold**. It takes the nearest one by geodesic distance. Levels 1 to 4 use finite scalar quantisation. The encoder's feature is squashed to $[-1, 1]$, rounded to a grid of three values per axis, and read out through a fixed orthonormal frame:

$$z = Wx,\ y = \tanh(z) \cdot \frac{L-1}{2},\ \hat{y} = \mathrm{round}(y),$$

Gradients pass through the rounding by the straight-through estimator, $\hat{y} + (y - \mathrm{sg}[y])$. The forward value is exact, and the backward pass behaves as if rounding were the identity. An odd number of levels per axis is what guarantees the exact zero that the prefix property needs.

An exponential moving average is used to maintain the codebook. Its target is the Fréchet mean: the point that minimises the sum of squared geodesic distances. It is not the arithmetic mean of the coordinates followed by a projection. On the outer shells, the two differ by more than two units of distance and the naive version silently removes codes from the manifold.

The five levels, as they are set in the delivered model

Read out of the shipped checkpoint rather than from the config, because the two can disagree and only one of them is the model:

Level	Quantiser	Alphabet	Grid axes	Aperture $\|v\|$	Shell radius
L0	geodesic codebook on $\mathbb{H}^{16}$	128	—	—	0.924 to 1.940
L1	FSQ, 3 values per axis	729	6	0.542	$\bar{r} + 0.15$
L2	FSQ	81	4	0.666	$\bar{r} + 0.30$
L3	FSQ	9	2	0.869	$\bar{r} + 0.45$
L4	FSQ	9	2	1.130	$\bar{r} + 0.60$

The radial step is $s = 0.15$ at every level and the mean L0 radius is $\bar{r} = 1.626$. The aperture schedule is not arbitrary, and what it holds constant is angular. On a shell of radius r a geodesic step of length d subtends an angle of about $d/\sinh r$, and $\sinh r$ grows with depth because every level is pushed one step further out. Measured at this checkpoint, the aperture increases by 2.08 from L1 to L4, while the angle increases by only 1.31. The angular reach would fall by 37% by L4. This is if it is held at L1's aperture. The deepest

levels would be the most tightly cornered. These are the levels that write individual bases. The schedule pays for the circumference.

The bit budget, and the honest reading of it

level	cells	bp per cell	alphabet	bits/cell	bits
L0	25	32	128	7.0000	175.0
L1	50	16	729	9.5098	475.5
L2	100	8	81	6.3399	634.0
L3	200	4	9	3.1699	634.0
L4	400	2	9	3.1699	1268.0
				total	**3186.5**

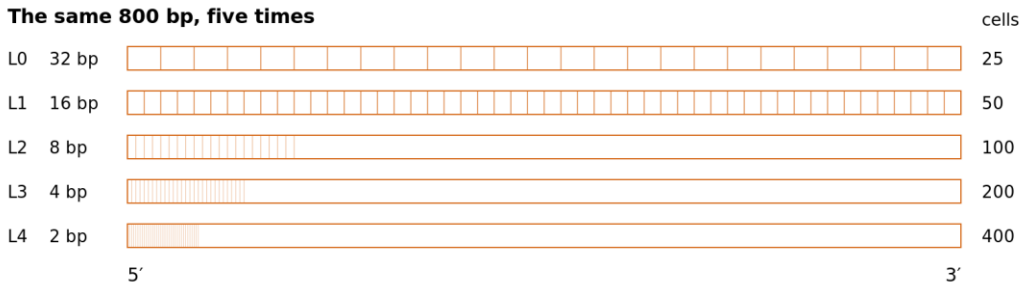


Figure 5: Each level covers the entire promoter at a different resolution, ranging from one symbol per 32 bases to one per two. This is why silencing a level provides information about a length scale rather than an anonymous channel. The 3' end is the base immediately before the start codon. Only the first stretch is drawn on the three finest levels, because 400 dividing lines across this width would appear as a solid block.

An 800 bp sequence carries bits. The ladder uses 3,186. This is 1.99 more than the sequence contains. We report this ratio alongside every claim about reconstruction. The two must be read together. With twice the budget, the latent sequence describes the promoter rather than abstracting it. Reconstruction from codes alone is essentially perfect, as follows from the budget. This is not a separate discovery.

The right word for this representation is '*auditable*'. You can see exactly which symbol corresponds to each of the 32, 16, 8, 4 or 2 bases. The wrong word here is "interpretable", which makes it look like the hierarchy has found grammar. There has been no such occurrence.

5. What the heads predict

All four heads read the same summary of the ladder from the pool. For each level, we calculate the mean, standard deviation and high-quantile statistic of its codes. That makes

240 numbers in total. The condition is fused into that vector by feature-wise modulation before the heads see it.

Strength, with its own error bar

The regression head emits three things: a mean μ, the log-variance of its own error, and a separate prediction of the *measured* replicate spread.

Training a heteroscedastic Gaussian the obvious way destroyed the run in three epochs. The loop is easy to see. Lowering σ on easy examples raises the relative weight of hard ones. Their σ must then grow, which lowers the easy ones further.

β-NLL damps that loop. It detaches a power of the variance:

$$\mathcal{L}_i = \mathrm{sg}\left[\sigma_i^{2\beta}\right]\left(\frac{(y_i-\mu_i)^2}{2\sigma_i^2} + \frac{1}{2}\log\sigma_i^2\right), \beta = 0.5.$$

A second exponent on the μ path is set so the gradient reaching the mean is exactly the mean-squared-error gradient. The variance is then an output we read. It is never a lever the model can pull to cut its own loss on hard rows. The two uncertainties are not tied, and that is on purpose. One says how wrong the model is. The other says how noisy the measurement was. Fusing them would make the training weight a function of a label.

Which of these two is stronger

The ranking head scores each promoter alone and learns on the difference. The model is Bradley–Terry:

$$P(A \succ B) = \sigma\left(\frac{s_A - s_B}{T}\right), \mathcal{L} = \mathrm{softplus}\left(-\mathrm{sign}\cdot\frac{s_A - s_B}{T}\right).$$

There is no any margin. This is a deletion, not a default. Once a pair is ordered past its threshold, a hinge loss has exactly zero gradient. Therefore, it stops learning while it is still incorrect about the size of the gap. The size of the gap is precisely what a correlation score measures.

Pairs are drawn within one strain, condition cell. *'Which is stronger'* is only a meaningful question when the conditions match. A reserved share of each batch holds the gene and condition fixed and varies the strain instead. Without these pairs, the strain channel receives no gradient from the head at all because the strain remains constant within every pair of the genome.

Is this even a promoter?

The discriminator distinguishes between genuine promoters and synthetic negatives that are matched in terms of composition. There are two types: dinucleotide-preserving shuffles and chimeras spliced from two promoters within the same genome.

The shuffle falsifies itself. It preserves composition exactly, so a GC-only baseline has no chance against it. The configuration refuses to build the head without that class. We added the chimera class after taking a measurement. A 5-mer linear probe could separate positives from shuffles with an area under the curve (AUC) of 0.80. However, with chimeras in the pool, the same probe drops to 0.41 against a real-versus-real band of 0.43.

Balancing three heads on one encoder

Three tasks share one encoder, and their gradients differ in scale by orders of magnitude. We rescale each task's gradient over the shared encoder to unit norm, multiply by an explicit prior, and renormalise:

$$\alpha_i = k \cdot \frac{\pi_i / \|g_i\|}{\sum_j \pi_j / \|g_j\|}, \pi = (1.5, 1.0, 0.6).$$

Reconstruction is **not** a fourth player. A Lagrange multiplier holds it at a target, updated multiplicatively, $\lambda \leftarrow \lambda e^{\eta(\text{target} - \text{EMA})}$.

The reason is arithmetic, not aesthetic. A bargaining solution equalises utilities. This gives the largest weight to the player with the weakest gradient, and a nearly solved task has the weakest gradient. We measured this on an earlier configuration. Reconstruction used 2.31 of a budget of 3.0, while the two prediction tasks were reduced to 0.39 and 0.31 respectively. The task's completion level was already at 0.9998.

Optimising on a curved space

Codebook points and strain vectors are points on the hyperboloid rather than free vectors. Therefore, they require a Riemannian optimiser. The Euclidean gradient is converted by raising the metric; with signature, this means flipping the time component. The update is an exponential map, not an addition. Momentum is parallel-transported to the new point. Using a standard optimiser on those tensors results in refusal rather than a warning.

6. How the data were split?

Twenty strains, roughly 26 000 orthogroups. A random split would put near-identical orthologues on both sides. So we group first, and split whole groups.

Split	Orthogroups	What it is for?
train	23 622	fits the weights
val	1 056	chooses an epoch *within* one run
spy	327	chooses *between* runs; opened once per comparison

Split	Orthogroups	What it is for?
test	1 322	opened once, at the end

VAL and SPY are distinct and have no homologues in common. This is more important than it sounds. The standard deviation of the evaluation statistic alone is 0.0064. Selecting a checkpoint under a metric that was noisy once produced a result that was 1.62 standard deviations above the truth. Selecting an epoch and a model on the same set inflates the number by a factor of two.

Both reference promoters are entirely contained within VAL: the weak target pks1 and the strong housekeeping tef1. They are never trained on. They are visible to epoch selection. They are absent from Spy and Test.

Epoch selection follows a written rule, applied in a fixed order. We established this rule before we saw any numbers. Two vetoes come first. The model must place **tef1** above **pks1**. Otherwise, the scale runs backwards at any correlation. Furthermore, single-base substitutions must improve the prediction beyond the measured noise floor in more than ten percent of cases. A model that is blind to single bases cannot be used for a design task with an eight-base budget.

We keep the epochs that SPY cannot tell apart within one bootstrap half-width of the best among the survivors. Then we select the highest val score from these. The band is the whole point. The maximum of a noisy statistic over thirty epochs is biased upwards, even when every epoch is identical.

The calibration is affine and is fitted to Spy, not Val. It does not alter any rank correlation. However, it does change the level, which is what users of the predictions need. The test split was read once for the delivered model. Its Spearman correlation is 0.689 ± 0.016 and its Pearson correlation is 0.692 ± 0.017 over 27,994 cells. Both bars are bootstraps over the 556 test orthogroups carrying a measured target, which are clustered because homologues of one orthogroup are not independent draws.

7. What each input is worth

A channel being connected is not the same as a channel being used, so we tested each one by taking it away from the trained model. We did that in two different ways, and it is the pair that tells you something rather than either number on its own:

- **Zeroing** replaces the channel with the model's learned "UNKNOWN" token, and asks what it loses without the information.
- **Shuffling** hands the model another row's value instead, and asks whether it was ever reading the channel row by row in the first place.

If a channel were used as a constant, zeroing it would be painful, whereas shuffling it would not. There are two columns because a single column would hide the answer. Pooled accuracy is calculated over every measured cell and is dominated by the easiest question: *"Which gene is this?"* Within-row removes that effect. It subtracts the average of each row across the conditions in which it was measured from both the prediction and the truth, and correlates what remains, thus only the part dependent on the growth condition is seen. This accounts for 8.2% of the variance across 5,003 rows.

removed	pooled	Δ pooled	within-row	Δ within-row	gap
nothing	0.69481	—	0.18680	—	+0.1298
experiment text	0.69321	−0.00160	0.18840	**+0.00160**	+0.1290
genomic flanks	0.68721	−0.00760	0.18444	−0.00236	+0.1342
gene features	0.66845	**−0.02636**	0.17835	−0.00845	+0.1517
gene **and** flanks	0.65295	**−0.04186**	0.17321	−0.01359	+0.1600
strain, *shuffled*	0.68866	−0.00615	0.16782	**−0.01898**	+0.1272
flanks, *shuffled*	0.67904	−0.01578	0.18356	−0.00324	+0.1371
gene, *shuffled*	0.66037	**−0.03444**	0.18516	−0.00164	+0.1469

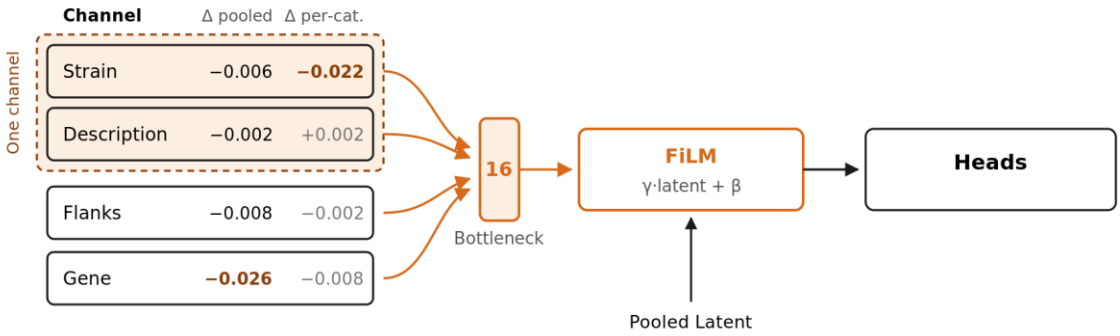


Figure 6. This measures the value of each conditioning channel by removing it from the delivered model. Δpooled shows how much overall accuracy is lost; Δper-cat shows how much is lost in the part that depends on the growth condition, which accounts for only 8.2% of the variance. The two columns disagree on which channel matters, which is why we report both. The gene features carry the overall score and the strain carries the condition-specific part. The dashed box marks a catch. Descriptions are stored per

genome and category, so removing one strain still leaves most of the same information supplied by the other. Taken together, they cost 0.012 overall and 0.025 for the condition-specific part, which is more than they add up to separately.

Five things follow, and the fourth retracts a claim this document used to make.

1) **Shuffling costs more than zeroing, on every channel.** Gene −0.034 against −0.026, flanks −0.016 against −0.008. That is the signature of a channel read *per row*. A wrong value is worse than a lack of knowledge, which is what the learned unknown token exists to say. Had the two been equal, the channel would have been acting as a constant and the model would not have been reading it at all.

2) **The ordering is gene, then flanks, then strain, at the end text.** Removing gene and flanks together costs **0.042**, more than removing each alone and adding the two up (0.034), so the two channels reinforce rather than substitute for each other.

3) **Removing conditioning makes the model overfit *more*, not less.** The gap beetwen train and validation split grows from 0.130 to 0.160 as channels are taken away, and the val column falls about four times faster than the train column. These channels are doing work on the validation side that the trunk otherwise has to invent from sequence alone.

4) **A missing description is free - a wrong one is not.** Replacing the text with its unknown token moves the within-row score by **+0.0016**. Handing the row *another row's* description costs **−0.0075**. The channel is being read: what the first number measures is that the unknown token is a harmless stand-in, not that the description is idle. Any claim about this channel built on the zeroing row alone is built on the wrong probe, and an earlier draft of this document made exactly that mistake in both directions.

5) **The per-category component is dominated by the strain, and only the within-row column shows that.** Shuffling the strain costs **−0.019 to −0.022** within-row across two runs against −0.006 pooled: it is the *largest* per-category effect of any channel while being nearly the smallest overall one. The two orderings genuinely disagree:

ranked by	first	second	third	fourth
pooled	gene	flanks	strain	text
within-row	**strain**	gene	flanks	text *(negative)*

This is a strain-by-category effect: in this strain, carbon-source experiments produce higher results than controls. The ridge baseline picks up the same structure when the one-hot strain takes its within-row score from 0.105 to 0.394. Reporting only the pooled table hid this completely.

The read-out's columns are not ablatable, which bounds the column. The cell readout generates its seven columns by projecting the category centroids through the shared text_proj. This means that the column directions are a fixed part of the head and cannot be removed by ablating a row's input. A channel can only reach this component through the pooled feature h, meaning that the within-row column measures how well a channel steers h along these fixed directions rather than whether it can create them.

The description and the strain are one channel, not two

Ablating them separately splits something that does not split, and the reason is in the artefact: the experiment-text cache is keyed `<genome>|<category>`, so the description is partly the strain. Measured on the cache the delivered run actually loaded, over its 85 keys and 79 distinct vectors:

factor the text vector is regressed on	η^2
(genome, category)	**1.000**, with 0 ambiguous cells
genome alone	**0.413**
control: genome labels shuffled	*0.218 ± 0.016*
category alone	0.216

The first row is the structural fact: the vector is a deterministic function of the cell, so the channel carries nothing a perfect (strain, category) pair would not. The second says the strain alone accounts for **0.41** of its variance, **+0.196** over the shuffle control. A single-channel ablation of either one is therefore scored while its partner is still supplying much of the same information.

So we removed them together, and read the interaction as (joint) − (sum of singles):

removed	Δ pooled	Δ within-row
description only, *zeroed*	−0.00160	+0.00160
description only, *shuffled*	−0.00224	−0.00754
strain only, *shuffled*	−0.00661	−0.02230
description + strain *(zero + shuffle)*	**−0.01160**	**−0.02482**
description + strain *(both shuffled)*	**−0.01011**	**−0.02810**

The strain was obscuring the value of the description. With the strain intact, zeroing the description costs 0.0016. If the strain has already been shuffled, the same zeroing costs 0.0050, which is 3.1 times more and the pooled interaction is −0.0034, i.e. super-additive. Shuffling instead of zeroing gives the same result: a factor of 1.6. When read as one

channel, the condition is worth approximately 0.010–0.012 when pooled and 0.025–0.028 within rows. These are the figures to quote for 'what the growth condition contributes', rather than the figures for the single-channel rows.

This is one place where the story does not generalise, as it is stated rather than smoothed. The masking result is clean in the pooled column for both probes. In the within-row column, the result holds for zeroing but reverses for shuffling. A shuffled description costs 0.0075 alone and 0.0058 once the strain has gone. This is slightly sub-additive. We do not have an explanation for this change in sign, and with the precision below, it is close to the noise of the probe.

The precision of the shuffle rows is important. The permutation is drawn from one stream, so a shuffled row depends on how many modes ran before it. Measured across two runs of the same checkpoint and strain, the results were −0.00615 and −0.00661 for shuffled, and 0.16782 and 0.16450 for pooled and within-row respectively: about ±0.0005 for pooled and ±0.003 for within-row. Differences smaller than this between two shuffle rows are not considered results. The zeroing rows are deterministic and carry no such spread.

Where this component goes against us

A strain-aware ridge captures more of variance of it than we do, and we would rather put that here than have it found. Scored on identical rows, with the model's better read-out:

within-row score	
model, sequence and strain only *(4 583 rows)*	0.178
model, every channel on *(5 003 rows)*	0.187
`kmer4+species_ridge`, sequence and strain only *(4 583 rows)*	**0.394**

The middle row is ours at its best; the first is the like-for-like one, since the ridge also sees nothing but sequence and strain. 2.2 more and it is not an artefact. We have checked the two most obvious escape routes, but neither is applicable: the row set contributes nothing (the component is 8.10% of cell variance on these rows, compared to 8.19% on the loop, and the target deviation is consistent to four decimal places). Turning every conditioning channel on only increases it by about +0.009. The difference between the two models lies in their shape. The ridge model has seven free per-category coefficient vectors. This model has none, generating its seven columns by projecting the category text through one shared projection. The architecture deliberately makes this trade-off free columns cannot describe a condition they have never encountered, but a text projection can and this number is the price it pays. Section 9 states what the whole table does and does not permit.

Here are two notes on how to read the table. Epochs 7 and 9 give the same pooled ordering with the same signs - the deltas move, but the conclusions do not, so none of this is noise from one epoch. The strain row is shuffled rather than zeroed on purpose: species_id = 0 is a real strain. The first in the registry so zeroing would measure “what if everything were strain B10” rather than “what does the strain contribute”.

What the architecture earns over something simpler

The ablation analysis indicates which inputs are important. However, it does not indicate whether the surrounding machinery was worth building. Therefore, we trained baselines using the same corpus, orthogroup-grouped split, seven-column cell target and metric. We selected every hyperparameter on VAL in the same way as the model's own epoch. The baselines consider only sequence and strain, and nothing else: no experiment text, no DNA-LM flanks, no orthology features and no motif bank. Every figure in the table is read on TEST, the split that neither the epoch nor the ridge penalty was chosen on. Full corpus: 27,994 test cells.

Model	Parameters	PCC	SCC
HYPPE, full architecture	4 506 780	**0.69231**	**0.68976**
CNN on one-hot + strain	114 503	0.60166	0.59420
CNN on one-hot	105 991	0.53419	0.51894
5-mer counts + strain, ridge	7 308	0.53340	0.55546
4-mer counts + strain, ridge	1 932	0.51489	0.53151
3-mer counts + strain, ridge	588	0.49384	0.51293
5-mer counts, ridge	7 175	0.46670	0.48153

4-mer counts, ridge	1 799	0.44254	0.45398
3-mer counts, ridge	455	0.41513	0.43727

K-mers are not collapsed onto their reverse complement because the model does not canonicalise them either, and using a baseline that did would answer a different question. The strain is the single largest factor by far. Adding a 19-way one-hot encoding shifts the ridge by +0.072 at k=4 and the CNN by +0.067. The overall expression of the nineteen genomes differs so much that knowing which genome is being analysed explains more than any additional sequence modelling that these baselines can provide. Any future results on this dataset must report whether they outperform strain-only information, because a model can appear effective by learning the genome.

Convolution is valuable, but only with full data. A subset would have given us the opposite result. On the full corpus, the CNN outperforms the best bag-of-k-mers by +0.083. When repeated on 4,000 orthogroups, the two tie (0.514 against 0.515), with the CNN using 59 times more parameters. Anyone reading an ablation study in this project should check what proportion of the data set the study was based on before accepting the results.

Having an output per category does not equate to knowing what the category is. When tested, the CNN without the strain scores −0.148 for the main effect of the condition. With seven free output columns and no input for the condition, its performance is worse than simply predicting the per-category mean. Adding the strain score increases this to +0.089. The ridges, whose per-category coefficients are explicit and linear, reach 0.283 on that axis, while HYPPE reaches 0.269. During validation, the order was reversed. Therefore, we cannot claim that the sentence embedding wins this axis. What the held-out split does support is that it is level with an explicit linear coefficient, not ahead of it. The margin is the least informative thing in the table either way.

When split onto the three axes of the task, one number becomes three movements in two directions: +0.056 for the gene, +0.099 for the condition and −0.081 for distinguishing the nineteen strains. The architecture is successful when the task involves sequence-to-expression and the growth condition has to be used, but it is unsuccessful when it comes to strain discrimination.

There are two numbers for one model, which are not interchangeable. The 0.69231 above is the delivered checkpoint read on TEST over the same 27,994 cells as the baselines. The 0.69481 against which the ablation deltas are measured is the same checkpoint on VAL, on which the ablations were run. The two differ by split, not by result: the deltas belong to the second number, while cross-model comparisons belong to the first. This project has often made the mistake of mixing them, which is worthy of mention.

The same architecture on enhancers

The baselines referenced above are the property of our project. We trained them using our corpus, so they can only answer the question of whether the machinery earns its place. The architecture itself is not specific to fungal promoters. The most obvious evidence of this is that the same hyperbolic ladder has been trained on a different regulatory element in a different organism against a published benchmark and competitor. In a related part of this codebase, the model reads Drosophila enhancers from the DeepSTARR assay (*de Almeida et al., 2022*). This comprises 402,296 training sequences, 40,570 validation sequences and 41,186 test sequences, each consisting of 249 bp and featuring two measured activities: developmental and housekeeping. The task is similar to ours in that it involves mapping sequence to strength. The element is an enhancer rather than a promoter.

It is worth stating what the published model is, since this is what is being measured against. DeepSTARR is a multi-task convolutional network adapted from Basset (*Kelley et al., 2016*). Four convolutional layers carry 256, 60, 60 and 120 filters of widths 7, 3, 5 and 3 respectively, each of which is followed by max pooling. This is followed by two fully connected layers of 256 units, then two outputs, one per activity class. Figure 7 illustrates this. The three stages it passes through describe the process that any model of this kind must follow: identifying what is present in the sequence, determining how those elements are arranged and converting this information into a task-specific number. We do not disagree about these three stages. It is about what sits between the second and third stages. Here, that is a discrete hyperbolic code. There, it is a flattened layer.

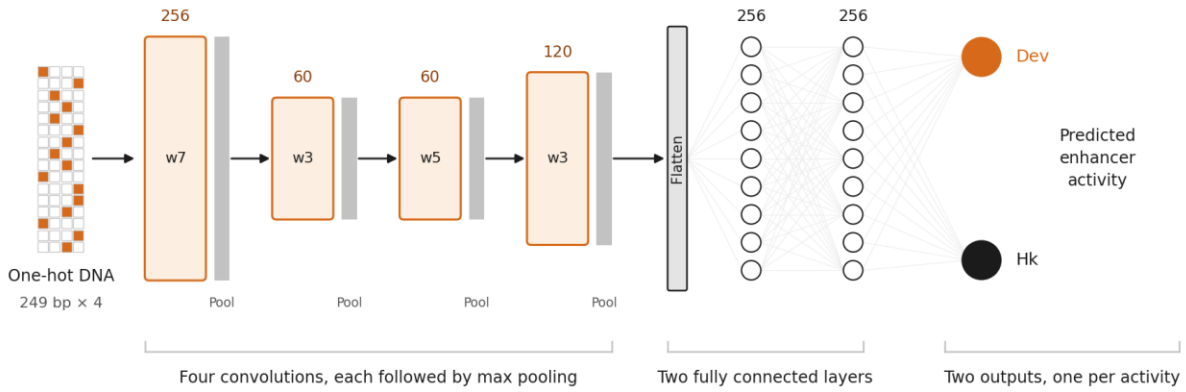


Figure 7. The DeepSTARR architecture, redrawn by us. One-hot DNA of 249 bp passes through four convolutional layers carrying 256, 60, 60 and 120 filters of widths 7, 3, 5 and 3, each followed by max pooling, then a flatten and two fully connected layers of 256 units,

and finally two outputs, one for developmental (Dev) and one for housekeeping (Hk) activity.

On the held-out test split of that benchmark, the ladder model achieves a Pearson correlation of 0.7244 for the developmental output and 0.7897 for the housekeeping output. This gives a mean of 0.7570 with 2.27 million parameters. In comparison, our implementation of LegNet (Penzar et al., 2023) achieved a mean of 0.7487, with 4.12 million parameters, reaching 0.7141 and 0.7832 on the developmental and housekeeping outputs, respectively. LegNet was the strongest available convolutional baseline for short regulatory regions, and it is the model that this family had been losing to. The margin is +0.0103 for the developmental output and +0.0065 for the housekeeping output, with 1.8 times fewer weights. When reverse-complement test-time augmentation is applied to both models, the margin narrows to +0.0039 and +0.0023. The same code was used to score both models through the same metric function on the same rows.

A third comparison is available and is the only one in this document that does not depend on anything we trained. This benchmark uses the published DeepSTARR model's own predictions for its test loci. Therefore, the model in Figure 7 can be scored alongside ours on identical rows. When deduplicated to 20,593 unique loci and reverse-complement averaging is applied, the published model achieves a score of 0.67572 for the developmental output and 0.74502 for the housekeeping output. Our model achieves scores of 0.72739 and 0.79242, representing increases of +0.0516 and +0.0474 respectively. Two cautions keep that honest. Firstly, it is a later checkpoint than the one in the paragraph above, and secondly, the loci are deduplicated, so those four numbers belong beside each other and not beside the table above. The gap is also against the 2022 architecture, not what has been published in the field since then.

The two architectures belong to the same family and differ in two respects. Both differences stem from the nature of the data. Firstly, the enhancer model runs three ladder levels over 249 bases instead of five over 800 bases, and it carries no conditioning stack because a DeepSTARR measurement has no strain, experiment description or flanking context. Its activity reads a trunk built on continuous motif occupancy, and the codes enter only as a projection detached from the gradient. There, latent decoding exists rather than prediction. Secondly, enhancer activity is symmetric under reverse complement. This model therefore ties its stem across the two strands and is trained using reverse-complement augmentation and test-time averaging. However, all three are forbidden here because a promoter is read in one direction only. Teaching this model strand invariance would therefore teach it something false.

What the model learned about the sequence itself

The ablations above say which inputs matter. They do not say what the model found inside the input that matters most. We asked that separately, by changing every base of a promoter in turn and recording how far each substitution moved the prediction, first on

pks1 and then on two hundred real promoters of the same strain. Three things came out of it, and each carries a control that has to fire before the result counts.

The first is a sense of position that nobody specified. A substitution matters more the closer it is to the start codon, roughly tripling in effect between the far end of the 800-base window and the last 160 bases. Nothing in the architecture provides this. The model is given a single positional ramp with no parameters and is never told that sequence near the gene should count for more; it learned the gradient from the data.

The second is a sensitivity to start codons upstream of the gene. Among nine three-letter words of comparable frequency in pks1, only ATG stood apart, and the effect came entirely from the four copies lying within 150 bases of the gene while the ten further upstream were unremarkable. Base composition does not explain it, because the anagrams of ATG sit at chance, and neither does the positional gradient, because the comparison is made against other windows drawn from each site's own neighbourhood. Across two hundred promoters the same preference holds over 438 different words containing ATG or its reverse complement, so it is a property of the model rather than of this one promoter.

That finding does not contradict our earlier one that the model has no start-codon mechanism. The earlier experiment inserted an ATG where there was none and saw nothing; this one destroys ones already present near the gene. Both are true, and the practical consequence runs opposite to the way we would have guessed: the design loop's hard rule against creating new upstream start codons is not made redundant by the model's own preferences, because the model was never trained to avoid creating one.

The third is a motif. Ranking every six-letter word by how much of the model's sensitivity sits beneath it, and comparing each word only against the other arrangements of its own letters so that base composition cannot manufacture the answer, the strongest preference the model holds is CACGTG. It comes first of the 159 arrangements of its own composition. It reads the same on both strands, and its two flanking variants are reverse complements of one another and rank tenth and fourteenth out of 3,770. The control matters here more than the result: the words that top the raw ranking are simply GC-rich, and none of them survives being compared with its own arrangements.

CACGTG is the E-box, the binding site of basic helix-loop-helix transcription factors. We did not go looking for it. It emerged from a ranking built only out of the model's own behaviour, with no motif database involved at any point. Chromatin-accessibility footprinting in Aspergillus niger independently recovered the same element among the five most enriched motifs in open promoter regions, where the genes beneath it were enriched for ribosomal proteins and translation, which is the most highly expressed class of genes a fungus has and exactly what a model predicting promoter strength has reason to learn. Sliding a six-base window along that ten-base footprint, the three windows that contain the core rank fifth, tenth and fourteenth, the fourth drops to 155th, and the fifth, which has lost the core entirely, falls to 2,915th. The signal decays exactly where the motif leaves the window, which is not a pattern chance produces.

What the model did not learn is as informative, and we can say why it could not have. The canonical regulators of fungal promoters are absent from the top of that ranking: carbon catabolite repression, the CCAAT complex, the xylanase activator and the light response elements all sit in the middle or the bottom of it. On pks1 in particular the light elements are neither present in excess nor weighed by the model, and that is the one thing its promoter was predicted to show, because pks1 is the gene required to make 6-pentyl-α-pyrone and that metabolite is produced in darkness and suppressed by light.

The reason lies in the training data rather than the architecture, which limits what this approach can deliver. Contrary to what we initially wrote, the corpus does not lack sequence variation. 61% of orthogroups appear in two or more strains, and only a third of gene promoter pairs are identical. Within the single **control condition**, nineteen strains share 15,930 genes, 88% of which differ in sequence. The paucity of coverage of the conditions that would teach a response is concerning. Light is measured in only three of nineteen strains and in 144 of 2,332 samples. Furthermore, the only contrast available for learning one is confounded by the fact that two strains differ in their regulators as well as in their promoters, and the strain channel exists to absorb exactly that. In order to learn which base makes a promoter responsive, there needs to be variation at a fixed genetic background, such as a mutagenesis library in one strain measured under both conditions.

Each of the negatives above sits behind a positive control that fires, so they are measurements rather than silence: on the same promoter and with the same null, the most sensitive five per cent of positions beat chance by about nineteen standard deviations. The full analysis, with its controls and the code that produced it, is the interpretability notebook in the repository (model/notebooks).

8. Choosing the ten promoters for the wet lab

The model is a predictor, but a panel is a decision. This section is how we got from one to the other.

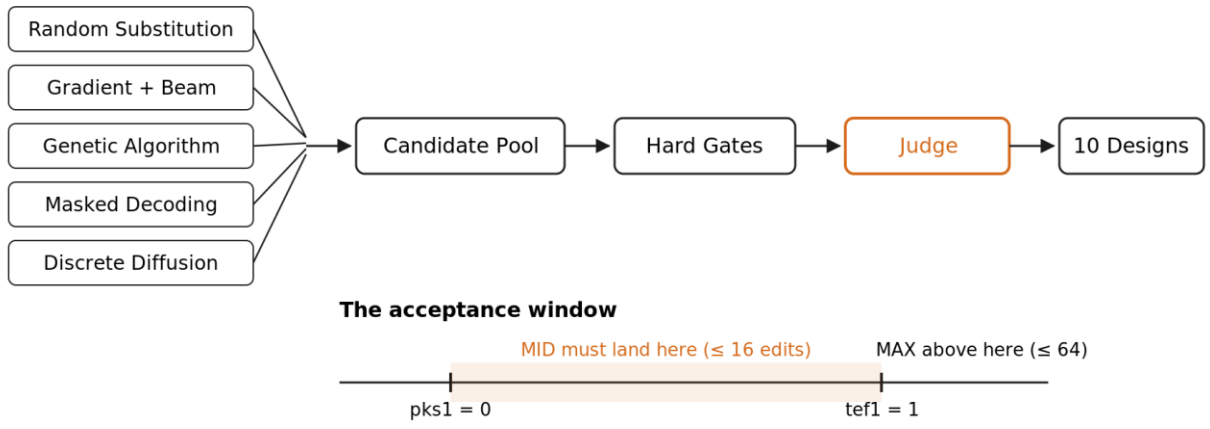


Figure 8. Five strategies compete with an identical budget of 30,000 model evaluations from a single fixed seed. Each strategy hands the judge the entire pool of candidates it has

collected along its path, rather than just its own best candidate. The Hard Gates are absolute: no ATG within the last 10 bp; GC and the longest homopolymer must be within the band, as measured on real promoters from the same genome; and there must be a minimum realness score. The Judge is a different model to the one proposed and has been fine-tuned using anchor pairs. Acceptance is two-sided, so a MID design can fail if it is too strong. The window is drawn in anchor units, with pks1 at 0 and tef1 at 1, but is set on the calibrated scale. The ten designs are provided with both native anchors alongside them.

Two anchors are used to set the scale: pks1, which is the promoter that we want to replace, and tef1, which is a strong constitutive promoter from the same strain. We report every prediction in anchor units, where 0 is **pks1** and 1 is **tef1**. This is because an absolute predicted value has no biological unit attached to it, and anchor units require no trust in our scale.

Reporting and deciding are not the same step and use different scales here. Anchor units are derived from the model's predicted difference between the two anchors. However, this difference is inaccurate: the model places the anchors much further apart than the measurement indicates. Consequently, a promoter can appear to be comfortably between the selected anchors units while, in reality, it is nowhere near the middle. Therefore, the acceptance window is set on the calibrated scale, not in anchor units. Cycle D6 on the Engineering page provides the measurement that necessitated this distinction.

There are two design briefs. The first, from MID, asks for a promoter between the two anchors, with a maximum of 16 bases changed out of 800. MAX asks for one above tef1 with a maximum of 64 bases. The budgets are small on purpose. A surgical edit will keep everything else about the promoter intact. This makes the result more interpretable and the construct more reliable.

We have developed five strategies with the same, one budget. The guided method has to beat random substitution. The other four strategies are: gradient-guided editing with beam search; genetic algorithms; masked iterative decoding; and discrete diffusion. Each strategy received 30,000 model evaluations from one fixed seed. A previous comparison had provided them with between 1,920 and 30,000 evaluations, so this comparison was measuring computing power rather than method.

The genetic algorithm deserves a paragraph to itself because its representation makes the edit budget reliable. An individual is not a sequence. Instead, it is a set of (position, base) substitutions relative to pks1. The budget is the length of the individual and cannot be exceeded by a penalty term; rather, it is exceeded by construction. Crossover is uniform across the two parents' sets, preserving the shared core. Mutation adds, removes or moves one substitution. This method took two of the ten available slots.

Scoring occurs on decoded DNA rather than codes. This is a measured constraint, not a preference. Substituting the entire coarse level significantly shifts the code-space prediction without altering a single base. Substituting at the finest level, however, changes

hundreds of bases and barely moves the code-space prediction. A search guided by the code-space score would optimise a number that the acceptance gate never sees.

The modeller and the judge are two different roles. The regression head makes the proposal. A ranking head, which is fine-tuned using anchor pairs, decides on membership. In the raw model, the "*is this above pks*1" decision was almost random, with balanced accuracy of 0.5669 on the 299 P1 validation promoters. The model was trained to order promoters within families and across the corpus. However, it was never trained on the anchor comparison accepted by the panel.

On validation, P1	before	after
Balanced accuracy, "above pks1"	0.5669	**0.8138**
Spearman on the held-out pks1 orthologues	−0.4857	**+0.8857**
Spearman on the corpus as a whole	0.5688	0.5135
Fraction of validation below tef1	0.9900	0.9967

This panel was accepted on the basis of a coin toss, with a boundary of 0.81. The family ranking went from backwards to +0.89. The batch retains a quarter of its pairs for family comparisons so as not to lose any gains. The penultimate row was paid for, and the general corpus ranking slipped by 0.055. This is the trade-off, and the reason there are two models rather than one is because of the distinction between the 'proposer' and the 'judge'. Their agreement on the chosen point is therefore a result rather than an assumption. Across the ten designs, the rank correlation was 1.00, with only one adjacent swap.

Every candidate must pass the hard gates. There must be no ATG in the last 10 bases, as this region is the 5' UTR immediately before the start codon and the presence of an ATG would create an upstream open reading frame. The GC content and the longest homopolymer must lie within the range observed in the promoters of the same strain. The discriminator provides a realness floor. The count of upstream ATGs further from the start is reported, but not enforced. In order to identify an ATG as a uORF, it is necessary to determine where transcription begins; however, this has not been established for this locus.

What was shipped included the part that did not go to plan. There are ten designs: five methods, each with two briefs. Both native anchors move within the panel, enabling the Wet lab to position everything according to the scale defined by the anchors. Six of the ten met the edit budget. Three of the MID designs changed five, 16 and 16 bases, while three of the MAX designs changed 59, 63 and 63 bases. These designs originated from gradient-guided editing, the genetic algorithm and masked decoding.

How the ten were made. All five methods optimise the same objective, the strength the model predicts under both constructs, and they differ in what they are allowed to change and in which space they search. Random substitution picks positions uniformly and swaps the base for another; it is the control, and it is in the panel because a comparison that quietly drops its control is not a comparison. Gradient editing relaxes the one-hot sequence into a continuous one, follows the objective's gradient back through the network using a straight-through estimator, and hardens the result to discrete bases at the end. The genetic algorithm keeps a population of edited copies of pks1, scores them and recombines the best, searching directly in base space, which is why a budget constrains it naturally.

The last two work in the model's own code space rather than in bases. Masked decoding hides a subset of ladder cells, decodes them again from the prior and repeats, each round conditioned on what the previous round settled. Discrete diffusion is the absorbing-state form of the same idea: it walks the codes to a fully masked state and learns the path back. Both then have to be turned into DNA, and that is where the two pieces of shared machinery matter more than the choice between the five.

The first decides where the edit goes. Spending the budget at random wasted most of it: at forty bases out of 800, random placement reached 0.2194 where a greedy oracle at the same budget reached 0.4806. So we built a sensitivity map over the 400 cells of the finest ladder level by resampling each cell on its own and keeping the best gain it could produce. 272 of the 400 cells help at all, the best is worth 0.0439, and they are not spread evenly along the promoter but cluster between bases 460 and 600. The budget is then spent where the map says an edit will do something.

The second is the splice, and it is the one that changes the molecule rather than the score. Decoding pks1's own codes and deliberately changing nothing returns eight wrong bases out of 800, because the round trip through the ladder is not quite lossless. The prediction hardly notices, moving by 0.0006 anchor units, but a synthesis house makes exactly the string we send it, so those eight would have been eight unintended mutations in a molecule the report calls pks1 with a local edit. Both code-space methods therefore splice their edit back into the original sequence, so that every base we did not mean to touch is the base that was there before.

The budget itself came from a measurement rather than a preference. Scoring all 2400 single substitutions and then applying the best K of them together gives an upper bound on what K edits can possibly buy: five edits reach the 5th percentile of real P1 promoters, ten reach the 25th, and twenty reach the median. Past roughly two hundred edits the objective falls again, so more editing is not more strength. The budgets sit inside that curve, at about two per cent of the sequence for MID and eight per cent for MAX, and the gate checks the edit count before it ever looks at the strength.

The other four did not, and the panel labelled them as such. Random substitution and discrete diffusion never met the brief within the budget, so they are delivered in an unconstrained second regime and are labelled *MID_free* and *MAX_free*. These four designs

changed 612, 580, 578 and 577 bases, retaining 23-28% identity with pks1. At this level of difference, they cannot be considered edits of pks1 in any meaningful sense; they are new sequences that simply originate from it. They are as far from the target as tef1, which differs at 616 positions.

We include them because a comparison of five methods that quietly drops the two that failed is not a fair comparison. One of the two failed methods is the random control, and a control that cannot meet the brief is not a useful control. The other is discrete diffusion, which generates a sequence rather than editing an existing one. Therefore, a budget of sixteen bases is not a natural constraint for it. We record this as an observation, not a diagnosis. We did not run the experiment that would distinguish between "the method cannot do this" and "our implementation of it could not".

The panel as delivered

Strength is given in anchor units, where **`pks1`** is 0 and **`tef1`** is 1. The last column is the score of the ranking head, the judge, which is a separate model from the one that proposed the sequence.

Design	Method	Brief	Edits	Identity to pks1	Strength	Judge
`pks1`	native anchor	n/a	0	1.000	0.000	0.31
`tef1`	native anchor	n/a	616	0.230	1.000	10.75
MID, gradient	gradient + beam	MID	16	0.980	0.694	5.52
MID, masked	masked decoding	MID	16	0.980	0.492	0.83
MID, genetic	genetic algorithm	MID	5	0.994	0.011	0.63
MAX, gradient	gradient + beam	MAX	59	0.926	1.121	12.10
MAX, masked	masked decoding	MAX	63	0.921	1.106	11.82
MAX, genetic	genetic algorithm	MAX	63	0.921	1.101	11.72

Design	Method	Brief	Edits	Identity to pks1	Strength	Judge
MID, random	random substitution	MID, free	612	0.235	0.584	2.84
MID, diffusion	discrete diffusion	MID, free	580	0.275	1.060	10.68
MAX, random	random substitution	MAX, free	578	0.278	0.636	4.39
MAX, diffusion	discrete diffusion	MAX, free	577	0.279	1.055	11.46

There are three things in that table that are worth reading rather than skipping over. Two of the designs miss their own brief and both are in the free regime. MID requires a promoter between the anchors, whereas MAX requires one above tef1. The MID diffusion lands at 1.060 above tef1, while the random MAX lands at 0.636 below it. Every design that adhered to the edit budget met its brief.

The genetic MID is technically inside the window and practically on the floor. It changed five bases and moved the prediction to 0.011 anchor units, which is pks1 with a rounding error. It is included in the panel as the smallest edit that the judge accepted, rather than as a design that we expect to perform differently to the native promoter.

The proposer and the judge agree on this, which is a result rather than an assumption. Ranking the ten designs according to the proposer and the judge produced the same order, except for one adjacent swap between the two diffusion designs. As they are separate models, the agreement is evidence that acceptance is not an artefact of whichever head happened to score.

Each design is scored twice, once for each construct. Our wet lab builds the panel in two plasmid contexts: pAN7-1 and pHYPPE. The model/data/final_promoters.tsv file contains a separate mean and standard deviation for each context. The table above shows the pAN7-1 column.

The two contexts differ by no constant: pHYPPE sits lower for the nine stronger rows and slightly higher for the two weakest. Therefore, the second context compresses the panel towards its middle rather than shifting it. We report this as an observation about the model rather than the plasmids: both contexts reach the model via the same 'unknown' flank token, so the difference lies in the model's uncertainty regarding an out-of-distribution neighbourhood rather than in its knowledge of either vector. The constructs and the measurements that will test these predictions belong to the wet lab pages rather than this one.

9. What these numbers claim, and what they do not

We would rather state these ourselves than have them discovered. The predictions are not based on fluorescence. There are three domain shifts between our training label and a wet read-out: mRNA to protein; native locus to plasmid; and rank within a library to one absolute value. The model was not trained on any of these.

The absolute scale is incorrect where we have been able to check it. Pks1 is measured at the 94th percentile of the promoters of its strain. However, our model places it near the 13th percentile. 'Stronger than pks1' therefore means stronger than the position assigned to pks1 by the model. This is precisely why a judge who has been fine-tuned on that boundary decides on membership, and why both anchors travel with the panel.

MAX is extrapolation. Asking for a promoter above the strongest anchor requires a prediction outside the range on which the judge was calibrated. The panel table labels those rows as such. The levels are not equal partners. The coarse level dominates the prediction and never reaches the decoder. The fine levels construct the sequence and barely affect the prediction. Equalising that leverage is the main open problem we are carrying forward.

The latent layer describes rather than abstracts. At 1.99 more the sequence's own entropy, the hierarchy carries the promoter almost literally. This is beneficial for decoding. However, it undermines any claim that the model has discovered regulatory grammar.

While there is a limit to what this corpus can teach a model about condition-specific regulation, it is not the limit initially proposed, and this correction is worth noting. A shortage of sequence variation is not the problem: 61% of orthogroups appear in two or more strains, and only a third of promoter pairs of one gene are identical in two strains. Within the single condition of control, nineteen strains share 15,930 genes, 88% of which differ in sequence. The problem is that conditions which would demonstrate responsiveness are underrepresented, and the available contrasts for learning about them are confounded. Light response is measured in three of the nineteen strains and in 144 of the 2,332 samples, and mechanical injury is measured in one strain. Asking which base makes a promoter responsive by comparing strains means comparing genomes that differ in their regulators as well as their promoters. This means that a promoter difference carries everything else that distinguishes the strains with it, which is exactly what the strain channel exists to absorb. The interpretability analysis shows this consequence: the model favours a motif associated with constitutively high expression and ignores light response elements on a gene whose metabolite is produced in darkness. In order to learn the responsive bases, variation is required at a fixed genetic background from a mutagenesis library in one strain, which is measured under both conditions.

Design time is zero-shot. No measurements from the target strain were entered into the pipeline. The strain enters via its learned position on the phylogeny. The condition enters via its written description. The plasmid context enters via a learned *UNKNOWN* token

because the plasmid neighbourhood sits 5.1 standard deviations outside the distribution of genomic flanks that the model encountered during training.

10. What the engineering cycles taught us as a method

Five engineering cycles sit behind this model. Each one begins with a measurement that exposed a problem, and each one ends with a change we made because of it: three changed the architecture, which is the code budget that could not carry the sequence, the strain channel that was structurally unable to express the contrast it existed for, and the hierarchy that was nominal until a radial step made it real; two changed how the model is judged rather than how it is built, which is the ranking loss that stopped learning while it was still wrong about the size of a gap, and the zero-shot failure at the one locus the project is about. Two of the five are visible on this page without leaving it: section 7 is the ablation that retracted a claim this document used to make, and section 8 is the failure at pks1 together with what the repair cost. The cycles are written out in full, with their designs, their measurements and their falsifiers, as cycles D3 to D6 on the Engineering page. What follows here is what they have in common, which is the part we would hand to another team.

Three failure modes recur, and all three fail silently without raising a runtime exception. The first is a regime or distribution mismatch: a threshold calibrated on random noise applied to real biological sequence, a baseline metric from a similar architecture misattributed during evaluation, or a training-mode statistic, with dropout active and batch statistics updating, compared against an inference-mode one. The second is a disconnected computation graph: sub-networks initialised, configured and allocated in memory that never receive a gradient or enter the forward pass, such as the gene encoder, the decomposition heads, or a configuration key silently overwritten by the parser. The third is a vacuous validation gate: a diagnostic that passes trivially, either because its threshold is already satisfied by random initialisation or because a bug in the assertion forces it to return true.

In response we changed how the codebase is tested, and this is the standard we would carry into any future architecture. Every module is verified with automated gradient-flow and graph-connectivity tests before training, so that no parameter is dead or detached from the loss, and a difference of exactly zero between a module switched on and switched off is treated as a failure signature rather than as a rounding result. Validation gates are themselves tested against synthetic inputs designed to force a failure, because an assertion that cannot fail is not a test. And all calibration metrics are computed strictly in evaluation mode on the target distribution, since a network in training and the same network at inference are not the same mathematical function.

One naming habit came out of the same cycles and is worth stating on its own. The phrase “the gene channel” referred to two different measurements for a while: a ridge on phylogenetic features predicting the label, which reached $R^2 = 0.3163$ against a shuffled control of -0.0069 ± 0.0033 and is why we built the channel, and a ridge on codon usage features predicting the model's residual, which reached -0.1279 and says the opposite.

Different features, different targets, opposite conclusions, one name. Every such number in this project is now qualified by what it predicts.

11. Reproducing this

Everything above runs from one seed, and the repository it runs from is https://gitlab.igem.org/2026/software/uw-poland/hyppe, with the model itself under /model. The model, the training loop, the split builder, the synthetic negatives and the diagnostics are all there, and so is the pipeline that rebuilds the training corpus from primary accessions. The two checkpoints are archived separately, at https://doi.org/10.5281/zenodo.23461451, and get-weights.sh fetches them and verifies each against its recorded SHA256 before anything loads them. bash examples/smoke/run.sh trains a small model on a synthetic corpus and runs the acceptance suite, with no data and no GPU required. The suite's strongest check compares 174 evaluation metrics before and after a checkpoint round trip, at tolerance zero.

The training corpus itself, meaning the promoter sequences paired with their expression values, is not versioned in the repository, because it is large. The pipeline that builds it from primary accessions lives in the same repository as the model, so the corpus can be rebuilt rather than taken from a pre-packaged table at face value, and the quantification choices behind every label stay inspectable. The derived tables are also deposited, at https://doi.org/10.5281/zenodo.23450836, with the per-run Salmon quantifications at https://doi.org/10.5281/zenodo.23450838, for anyone who would rather start from them than rerun the pipeline.

What ships with this repository

Enough to rebuild the delivered model and to check every claim on this page without asking us for anything.

trained weights	model/weights/, filled by get-weights.sh from https://doi.org/10.5281/zenodo.23461451 and checked against the SHA256 recorded in WEIGHTS.txt
the motif bank	`model/data/motifs.meme`, all 252 frozen profiles
the split	`model/data/splits/`, every orthogroup assignment
the delivered panel	`model/data/final_promoters.fasta` and `.tsv`
conditioning artefacts	the 20-leaf tree, the condition taxonomy, the category text vectors

orthology tables	`model/data/final_datasets/orthology/`

Which checkpoint every number on this page came from

Two checkpoints ship, and they do different jobs, so it is worth saying plainly which one each figure was measured on.

Checkpoint	What it is	What we report from it
`hyppe_foundational.pt`	The foundation model, epoch 8	Everything except the panel: the bit budget and ladder geometry in section 4, the strain space in section 3, all of section 7, and the test score of 0.689
`hyppe_judge.pt`	The same model with its ranking head fine-tuned on anchor pairs	The acceptance decision for the ten designs in section 8, and the four boundary figures there

The judge genuinely shares the same trunk as the foundation model. We did not take this on trust from the training script. We compared the two checkpoint files, tensor by tensor. Out of 471 tensors, 465 are identical bit for bit and the six that differ belong exclusively to the ranking head, specifically its two trunk layers and output layer. Nothing outside of this head moved, which is precisely what 'the judge and the model share an encoder' must mean if the panel's acceptance is to be meaningful.

If you inspect the files yourself, two technical details are worth noting. Firstly, the judge still carries its parent's name and epoch in its top-level metadata, alongside the foundation model's training metrics. Consequently, the two files cannot be distinguished by their primary headers alone, but rather by their checksums or the six tensors described above. The judge's own fine-tuning metrics are stored under a separate dictionary entry, which also records the random seed and the step at which the run stopped. This entry verifies that the weight change outside the ranking head was exactly zero.

Secondly, take care when reading the script's source header rather than inspecting the checkpoint file directly. The training script's documentation quotes boundary figures from earlier exploratory runs (rising from 0.5406 to 0.83 with a fall in family ranking from 0.8857 to 0.54) that do not represent the final checkpoint. All figures reported in the above table are read directly from the shipped weight file.

Finally, readers who want to execute the code immediately rather than rebuild the corpus first can run the bash `examples/smoke/run.sh` script to train a small model end-to-end

on a synthetic dataset and run the full acceptance suite, requiring no external data, downloads or GPU.

References

Bradley RA, Terry ME (1952). *Rank analysis of incomplete block designs. Biometrika* 39:324-345. doi:10.2307/2334029

Van den Oord A, Vinyals O, Kavukcuoglu K (2017). Neural discrete representation learning. *NeurIPS*. arXiv:1711.00937

Nickel M, Kiela D (2018). Learning continuous hierarchies in the Lorentz model of hyperbolic geometry. *ICML*. arXiv:1806.03417

Rezende DJ, Viola F (2018). Taming VAEs. arXiv:1810.00597

Becigneul G, Ganea OE (2019). Riemannian adaptive optimization methods. *ICLR*. arXiv:1810.00760

Wang W, Wei F, Dong L, Bao H, Yang N, Zhou M (2020). MiniLM: deep self-attention distillation for task-agnostic compression of pre-trained transformers. *NeurIPS*. arXiv:2002.10957

Navon A, Shamsian A, Achituve I, et al. (2022). Multi-task learning as a bargaining game. *ICML*. arXiv:2202.01017

Seitzer M, Tangemann A, Scholkopf B, Martius G (2022). On the pitfalls of heteroscedastic uncertainty estimation with probabilistic neural networks. *ICLR*. arXiv:2203.09168

Mentzer F, Minnen D, Agustsson E, Tschannen M (2024). Finite scalar quantization: VQ-VAE made simple. *ICLR*. arXiv:2309.15505

Johnson WE, Li C, Rabinovic A (2007). Adjusting batch effects in microarray expression data using empirical Bayes methods. *Biostatistics* 8:118-127. doi:10.1093/biostatistics/kxj037

Emms DM, Kelly S (2019). OrthoFinder: phylogenetic orthology inference for comparative genomics. *Genome Biology* 20:238. doi:10.1186/s13059-019-1832-y

Chen S, Zhou Y, Chen Y, Gu J (2018). fastp: an ultra-fast all-in-one FASTQ preprocessor. *Bioinformatics* 34:i884-i890. doi:10.1093/bioinformatics/bty560

Patro R, Duggal G, Love MI, Irizarry RA, Kingsford C (2017). Salmon provides fast and bias-aware quantification of transcript expression. *Nature Methods* 14:417-419. doi:10.1038/nmeth.4197

Cervantes-Badillo MG, Muñoz-Centeno T, Uresti-Rivera EE, Argüello-Astorga GR, Casas-Flores S (2013). The Trichoderma atroviride photolyase-encoding gene is transcriptionally

regulated by non-canonical light response elements. FEBS Journal 280:3697-3708. doi:10.1111/febs.12362

Moreno-Ruiz D, Fuchs A, Missbach K, Schuhmacher R, Zeilinger S (2020). Influence of different light regimes on the mycoparasitic activity and 6-pentyl-α-pyrone biosynthesis in two strains of Trichoderma atroviride. Pathogens 9:860. doi:10.3390/pathogens9100860

Huang L, Li X, Dong L, Wang B, Pan L (2021). Profiling of chromatin accessibility identifies transcription factor binding sites across the genome of Aspergillus species. BMC Biology 19:189. doi:10.1186/s12915-021-01114-0

Flatschacher D, Eschlböck A, Pierson S, Schreiner U, Stock V, Schiller A, Ruso D, Doppler M, Ruzsanyi V, Gründlinger M, Büschl C, Schuhmacher R, Zeilinger S (2025). Linking a polyketide synthase gene cluster to 6-pentyl-α-pyrone, a Trichoderma metabolite with diverse bioactivities. Microbial Cell Factories 24:89. doi:10.1186/s12934-025-02718-9

Ji Y, Zhou Z, Liu H, Davuluri RV (2021). DNABERT: pre-trained bidirectional encoder representations from transformers model for DNA-language in genome. *Bioinformatics* 37:2112-2120. doi:10.1093/bioinformatics/btab083

Zhou Z, Ji Y, Li W, Dutta P, Davuluri R, Liu H (2024). DNABERT-2: efficient foundation model and benchmark for multi-species genome. *ICLR*. arXiv:2306.15006

Liu F, Shareghi E, Meng Z, Basaldella M, Collier N (2021). Self-alignment pretraining for biomedical entity representations. *NAACL*. arXiv:2010.11784

Gu Y, Tinn R, Cheng H, Lucas M, Usuyama N, Liu X, Naumann T, Gao J, Poon H (2021). Domain-specific language model pretraining for biomedical natural language processing. *ACM Transactions on Computing for Healthcare* 3:1-23. doi:10.1145/3458754

Devlin J, Chang M-W, Lee K, Toutanova K (2019). BERT: pre-training of deep bidirectional transformers for language understanding. *NAACL*. arXiv:1810.04805

Kelley DR, Snoek J, Rinn JL (2016). Basset: learning the regulatory code of the accessible genome with deep convolutional neural networks. *Genome Research* 26:990-999. doi:10.1101/gr.200535.115

de Almeida BP, Reiter F, Pagani M, Stark A (2022). DeepSTARR predicts enhancer activity from DNA sequence and enables the de novo design of synthetic enhancers. *Nature Genetics* 54:613-624. doi:10.1038/s41588-022-01048-5

Penzar D, Nogina D, Noskova E, Zinkevich A, Meshcheryakov G, Lando A, Rafi AM, de Boer C, Kulakovskiy IV (2023). LegNet: a best-in-class deep learning model for short DNA regulatory regions. *Bioinformatics* 39:btad457. doi:10.1093/bioinformatics/btad457